DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Carry the Type

CMSC 124 Activity 5

Prepared by: Rene Andre Bedonia Jocsing

Published: September 10, 2026 **Deadline:** September 10, 2026

A context-free grammar can prove that `R = P + Q` follows the syntax of an assignment. What if you need to check if the assignment makes sense? An **attribute grammar** adds attributes, rules, dependencies, and predicates to that grammar so it can describe contextual facts such as types. Today you'll carry those facts through a syntax tree before a final predicate accepts or rejects the assignment.

The language here is invented for the activity. It's written to be read. Every rule it depends on is on this page.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discuss with classmates and the instructor at any point. This activity is worth 15 points. Nine points reward correct attribute constructions, six reward the theory that supports them.

The Type Rules

Our tiny language has two types, `int` and `real`, and one operator, `+`. Use only these rules:

1. An identifier's declared type comes from the symbol table.
2. `int + int` produces `int`. Any addition with at least one `real` operand produces `real`.
3. An assignment is valid only when its target and complete expression have the same type. This language performs no automatic conversion.

The symbol table for the running case is:

Name	Declared type
`P`	`int`
`Q`	`real`
`R`	`real`

An **attribute** is information attached to a node. Here, `actual` is the type an expression produces, while `expected` is the type its assignment context requires. A **semantic function** computes one attribute from the values it depends on. A **synthesized attribute** moves upward from children to a parent, so expression nodes synthesize `actual`. An **inherited attribute** arrives from a parent or sibling, so the assignment passes `expected` down to the expression root.

The final **predicate** is a true-or-false condition:

```
1 expression.expected == expression.actual
```

Read the predicate as "the expression's expected type equals the expression's actual type."

Don't evaluate that predicate until both sides are known. It runs once for the whole assignment, at the root. It doesn't run again at the + node or at either leaf.

The Rules Written Out

An attribute grammar is usually written as a syntax rule with its semantic rules stacked underneath it. That's the notation we'll use whenever this course writes attribute rules out. Today's assignment looks like this in that notation:

```
Syntax rule:    <assign> ::= <target> = <expr>
Semantic rules: <expr>.expected  <- <target>.actual      (inherited)
                <expr>.actual    <- typeOf(<expr>)       (synthesized)
Predicate:      <expr>.expected == <expr>.actual
```

Read `<-` *as "gets its value from." Read* `typeOf(<expr>)` *as "the type the addition rule computes from the operands below."*

`typeOf` is rule 2 and nothing more. With two types and one operator, the whole function fits in four lines:

```
typeOf(int, int)   = int
typeOf(int, real)  = real
typeOf(real, int)  = real
typeOf(real, real) = real
```

Only the first line produces `int`. One `real` operand anywhere makes the result `real`.

Each semantic rule says which attribute is being set and where its value comes from, with the parenthesized word recording which way the value travelled through the tree. Inherited means it arrived from the parent, and synthesized means it came up from the children. The annotated trees in the rest of this activity present these same rules as diagrams.

Worked Example for Checkpoint 1

Suggested time: 22 minutes, including the worked example.

Use a smaller symbol table in which `T`, `M`, and `N` all have type `int`, then analyze `T = M + N`. Everything is `int` on purpose, so the predicate comes out true and you can watch the machinery with nothing else going on. The checkpoint changes one thing, which is that one of the two operands is `real`.

1. Looking up `T` gives `int`, so the expression root inherits `expected = int`.
2. Looking up `M` and `N` gives `actual = int` at both leaves. Either lookup may happen first.
3. The addition rule receives `int + int`, so the + node synthesizes `actual = int`.
4. Both predicate inputs now exist. `int == int` is true, so the assignment is valid.

One valid schedule is `T`, inherited `expected`, `M`, `N`, addition `actual`, then the predicate. Swapping the `M` and `N` lookups gives another valid schedule because neither leaf depends on the other.

Dependencies control the schedule. The independent leaf lookups may occur in either order. Evaluating the predicate as soon as `expected` is known ignores its missing right side. Calling that comparison true misses an input.

The four figures below are one tree. Its structure never changes. The annotations arrive one obligation at a time until the predicate has both inputs.

Let attributes move through the tree

The syntax stays fixed while declarations supply leaf types and dependencies carry them to the predicate.

Start with syntax alone

The tree identifies the target and the addition, but no type judgment is available yet.

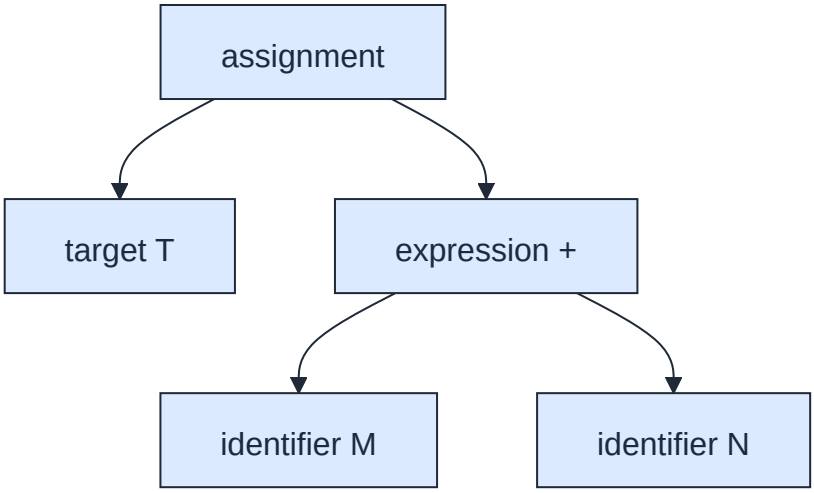

Figure 1: Start with syntax alone

Carry the target to expected

The target lookup supplies `int`, then the assignment passes that type down as `expected`.

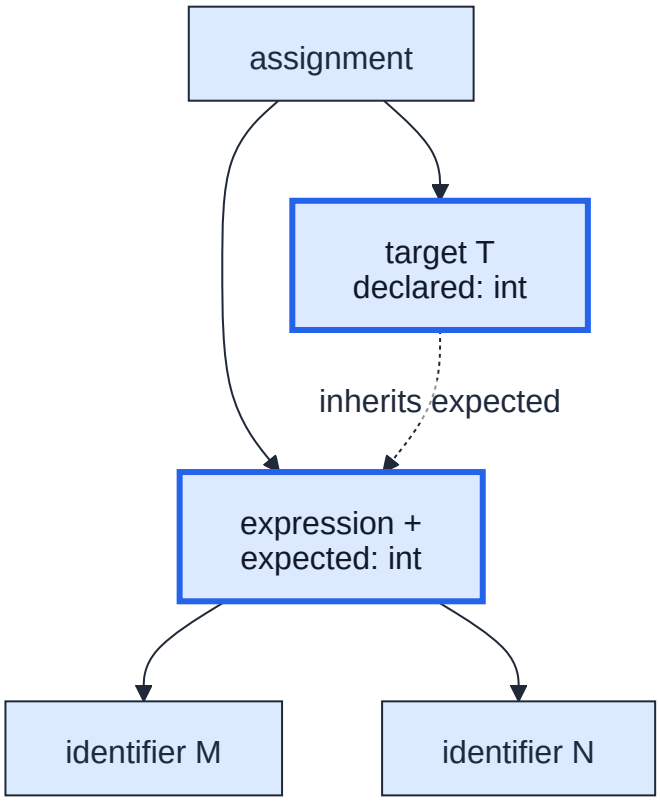

Figure 2: Carry the target to expected

Carry the operands to actual

The operand lookups flow up into the addition's `actual` type. The predicate now has both inputs.

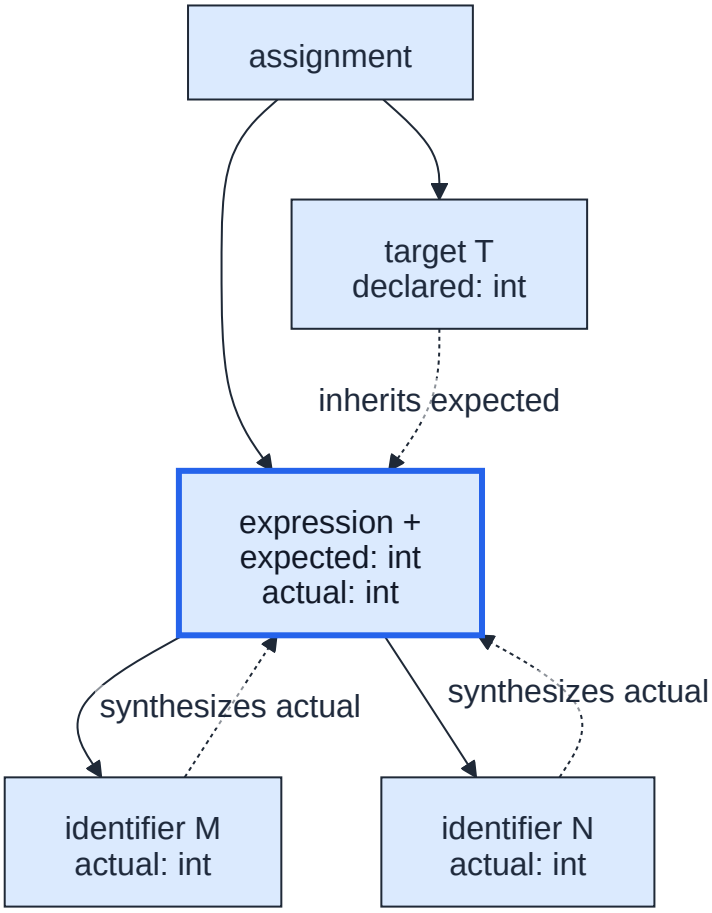

Figure 3: Carry the operands to actual

Evaluate the predicate

The predicate receives `expected = int` and `actual = int`, compares them, and accepts the assignment.

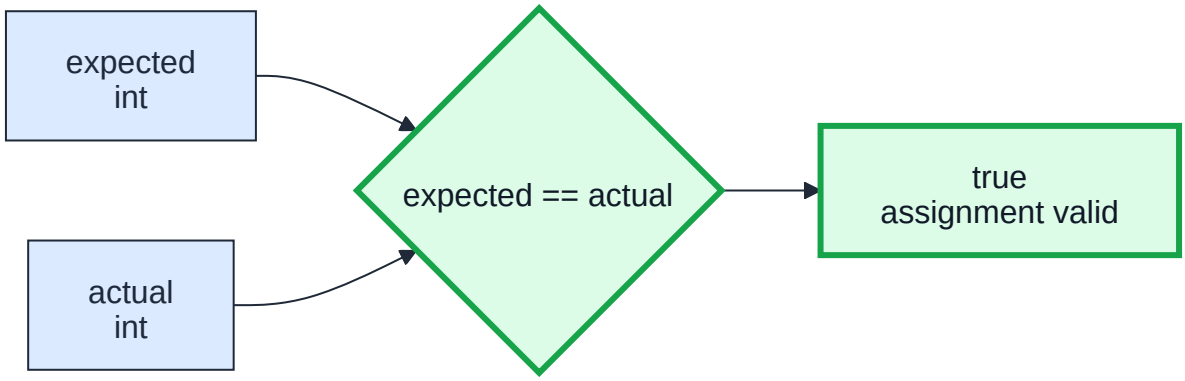

Figure 4: Evaluate the predicate

Checkpoint 1: Annotate the Tree (5 Points)

Running Case

Analyze this assignment with symbol table `P : int`, `Q : real`, and `R : real`:

```
R = P + Q
```

Draw `assignment` as the root, with `target R` and `expression +` beneath it. The expression has `identifier P` and `identifier Q` as children. Use the worked diagram as the layout model, but annotate this fresh case yourself.

The target type flows down as `expression.expected`. The operand types flow up to compute `expression.actual`. Here, `int + int` produces `int`, while any addition containing `real` produces `real`. Evaluate `expression.expected == expression.actual` only after both values exist.

1. (Construction: 4 points) Copy the tree and attach all of these values to the appropriate nodes: the target's looked-up type, the root's inherited `expected`, both leaf `actual` types, the + node's synthesized `actual`, and the predicate result.

2. (Explanation: 1 point) Explain why `expected` travels downward while `actual` travels upward. Your explanation must identify the node that supplies each value.

Dependency Comes Before Order

An **attribute dependency** says that one value must exist before another can be computed. It sets prerequisites without dictating a traversal order. For this case, the + node depends on both child `actual` values. The predicate depends on the root's `expected` and `actual` values.

Worked Example for Checkpoint 2

Suggested time: 16 minutes, including the worked example.

For `T = M + N`, one valid schedule is target lookup, inherited `expected`, lookup of `M`, lookup of `N`, synthesized addition `actual`, then the predicate. The two leaf lookups may swap or occur before the target lookup because they don't depend on each other.

Follow the attribute dependencies

Each arrow points from a value that must exist to the value that uses it. The target and operand lookups remain independent.

Respect the prerequisites

The predicate waits for both expected and addition actual, while addition actual waits for both operand types.

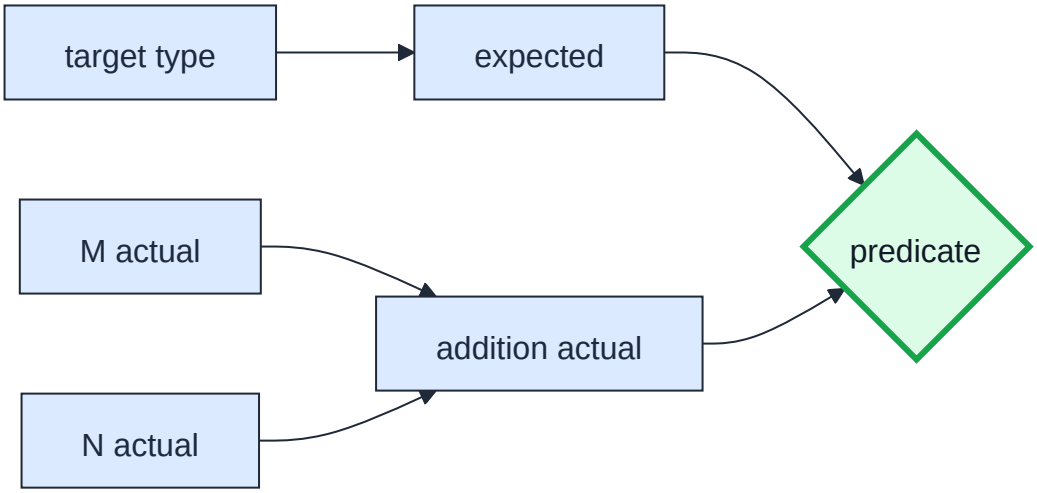

Figure 5: Respect the prerequisites

Read each arrow as "the value on the left must exist before the value on the right." Read `target type -> expected` *as "the target type flows to expected."*

The predicate remains last because both of its incoming values must exist. A fixed left-to-right rule is too strict. Dependencies permit several orders even though they forbid evaluating the predicate early.

Checkpoint 2: Schedule the Work (4 Points)

Schedule the attributes for `R = P + Q` with `P : int`, `Q : real`, and `R : real`. The target lookup supplies `expected`, both operand lookups must precede the addition's `actual`, and both `expected` and `actual` must precede the final `expression.expected == expression.actual` predicate.

1. (Construction: 2 points) Write one valid evaluation order for every attribute and the final predicate in `R = P + Q`. Draw all five dependency arrows.

2. (Explanation: 2 points) Explain why evaluating the predicate first would be a logical leap. Then give one different valid order, or explain why no different order exists.

Two schedules can both be correct when they respect the same dependencies. Full credit comes from accurate dependency arrows and a valid schedule.

Worked Example for Checkpoint 3

Suggested time: 22 minutes, including the worked example.

Suppose `I` and `J` are `int`, `K` is `real`, and the assignment is `I = J + K`. Its target supplies `expected = int`. Combining the operand types synthesizes `actual = real`, which makes the final predicate false.

Predicate input	Value
`expected`	`int`
`actual`	`real`
result	false

Every identifier has a declared type, but that only supplies the predicate's inputs. The predicate still has to check whether they agree.

Checkpoint 3: Let the Predicate Decide (6 Points)

Analyze `P = P + Q` with symbol table `P : int`, `Q : real`, and `R : real`. Work the attributes out from the rules in this activity, then finish by evaluating `expression.expected == expression.actual`. `R` is

still declared, so it stays in the table, but this assignment never mentions it. Don't hunt for a place to fit it.

1. (Construction: 3 points) Draw and annotate the complete tree, including every intermediate type and the predicate result.

2. (Explanation: 3 points) Point to where the `P = P + Q` tree first differs from the earlier `R = P + Q` tree. State the wrong conclusion that the expression is acceptable because every identifier has a declared type, then use the assignment predicate to correct it.

End with the completed tree and one sentence connecting its final judgment to the predicate. Don't skip from the leaf types straight to valid or invalid.